

Psico-RAG API — Documentação das Rotas

Contexto: Microserviço FastAPI que expõe recursos de busca semântica (RAG) e geração de respostas com/sem modelo supervisionado, integrando coleção pública (literatura) e coleção privada (prontuário) via Qdrant. Este documento é didático e inclui exemplos de requisição, retorno e explicações dos termos.

Sumário

- [Visão geral](#)
 - [Autenticação e Autorização](#)
 - [Cabeçalhos-Padrão](#)
 - [Códigos de Status e Erros](#)
 - [Rotas](#)
 - [GET /health](#)
 - [POST /search](#)
 - [POST /answer](#)
 - [Modelos de Requisição/Resposta](#)
 - [Exemplos completos \(cURL e Python\)](#)
 - [Boas práticas de uso](#)
 - [Glossário — Siglas e Teorias](#)
-

Visão geral

A API oferece dois fluxos principais: 1. **Busca semântica** (*Retrieval*) de trechos relevantes em coleções vetorizadas no **Qdrant**. 2. **Geração de resposta** (*Answering*) com base nos trechos recuperados (RAG) e/ou com **modelo supervisionado** treinado em dados rotulados (ex.: classificação por turma/risco), de acordo com o `mode` escolhido.

Modos de operação (campo `mode`)

- `rag` (*padrão*): usa somente RAG (LLM + trechos recuperados).
- `model`: usa o **modelo supervisionado** treinado (ex.: XGBoost/Sklearn) para ranquear/gerar respostas específicas de tarefa.

Observação: Em versões anteriores, chamávamos de “baseline” o fluxo sem prontuário. Neste doc, padronizamos como `rag` (sem notas privadas) e `rag` + `include_notes=true` (com notas, mediante papel autorizado).

Autenticação e Autorização

- **API Key:** obrigatória em todas as rotas de negócio.
- Header: `X-API-Key: <sua-chave>`
- **Papel do usuário (Role):** necessário **apenas** quando acessar coleção privada (prontuário) com `include_notes=true`.

- Header: `X-User-Role: psych` (ou outro papel autorizado pelo backend)

Cenários - Consulta pública (literatura): só `X-API-Key` . - Consulta com prontuário: `X-API-Key` e `X-User-Role` com permissão.

Cabeçalhos-Padrão

- `Content-Type: application/json`
 - `X-API-Key: dev-key` (exemplo)
 - `X-User-Role: psych` (apenas quando `include_notes=true`)
-

Códigos de Status e Erros

- `200 OK` : requisição válida; retorno no corpo.
- `400 Bad Request` : falta de parâmetros obrigatórios ou formato inválido.
- `401 Unauthorized` : API Key ausente/inválida.
- `403 Forbidden` : papel do usuário não permite acessar `include_notes=true` .
- `404 Not Found` : coleção/identificador inexistente.
- `422 Unprocessable Entity` : validação Pydantic dos campos.
- `500 Internal Server Error` : erro inesperado no servidor/modelo.

Formato de erro (exemplo)

```
{
  "detail": "Mensagem descritiva do erro"
}
```

Rotas

GET /health

O que faz: verifica se a API está de pé e, opcionalmente, se há conectividade com Qdrant/modelo.

Requisição: sem corpo.

Resposta (200)

```
{
  "status": "ok",
  "version": "0.1.0",
  "qdrant": "up",
  "model": "loaded"
}
```

POST /search

O que faz: executa **busca semântica** em coleções vetorizadas (ex.: literatura pública). Retorna os **top-k** trechos mais similares à `query`.

Corpo (JSON) | Campo | Tipo | Obrigatório | Padrão | Descrição | |---|---|---|---|---| | `query` | string | sim | — | Texto da busca. | | `k` | int | não | 3 | Quantos trechos retornar (top-k). | | `collection` | string | não | `literatura` | Nome da coleção no Qdrant. | | `include_notes` | bool | não | false | Se `true`, inclui busca na coleção privada (requer `X-User-Role`). | | `class_id` | string | não | — | Id/turma usada em filtros opcionais do payload. |

Resposta (200)

```
{
  "query": "Top 3",
  "k": 3,
  "results": [
    {
      "doc_id": "artigo_123.pdf",
      "chunk_id": "artigo_123_17",
      "score": 0.8421,
      "text": "Trecho relevante...",
      "metadata": {
        "source": "literatura",
        "page": 4,
        "tags": ["evasão", "fatores de risco"]
      }
    },
    {
      "doc_id": "artigo_456.pdf",
      "chunk_id": "artigo_456_08",
      "score": 0.8233,
      "text": "Outro trecho...",
      "metadata": { "source": "literatura", "page": 10 }
    }
  ]
}
```

Observações - `score` é a similaridade (quanto **maior**, mais parecido) ou distância convertida (dependendo da métrica configurada no Qdrant). - Quando `include_notes=true`, os itens retornados podem vir de `source="prontuario"`.

POST /answer

O que faz: gera uma **resposta em linguagem natural** a partir da `query`, usando RAG (trechos recuperados) e/ou **modelo supervisionado** de acordo com `mode`.

Corpo (JSON) | Campo | Tipo | Obrigatório | Padrão | Descrição | |---|---|---|---|---| | `query` | string | sim | — | Pergunta ou instrução do usuário. | | `k` | int | não | 3 | Quantidade de trechos a recuperar como contexto. | | `class_id` | string | não | — | Identificador de turma, se aplicável ao modelo. | | `mode` | string | não | `rag` | `rag` ou `model`. | | `include_notes` | bool | não | false | Se `true`, inclui prontuário no retrieval (requer `X-User-Role`). |

Resposta (200)

```
{
  "mode": "model",
  "query": "Top 3",
  "answer": "Resumo dos três principais fatores...",
  "citations": [
    { "doc_id": "artigo_123.pdf", "chunk_id": "artigo_123_17", "page": 4 },
    { "doc_id": "anotacao_aluno_91.parquet", "chunk_id": "91_2024_07",
  "source": "prontuario" }
  ],
  "diagnostics": {
    "retrieval_k": 3,
    "used_model": "xgboost_v1",
    "latency_ms": 512
  }
}
```

Observações - Em `rag`, a chave `used_model` pode ser `llm=<nome>`; em `model`, indica o classificador/regressor carregado. - `citations` permite rastrear a origem da resposta.

Modelos de Requisição/Resposta

Schema — `/search` (request)

```
interface SearchRequest {
  query: string;
  k?: number;           // default 3
  collection?: string;  // default "literatura"
  include_notes?: boolean; // default false
  class_id?: string;
}
```

Schema — `/search` (response)

```
interface SearchResponse {
  query: string;
  k: number;
  results: Array<{
    doc_id: string;
```

```

    chunk_id: string;
    score: number; // similaridade
    text: string;
    metadata?: Record<string, any>;
  }>;
}

```

Schema — `/answer` (request)

```

interface AnswerRequest {
  query: string;
  k?: number; // default 3
  class_id?: string;
  mode?: 'rag' | 'model';
  include_notes?: boolean; // default false
}

```

Schema — `/answer` (response)

```

interface AnswerResponse {
  mode: 'rag' | 'model';
  query: string;
  answer: string; // texto em linguagem natural
  citations?: Array<{
    doc_id: string;
    chunk_id: string;
    page?: number;
    source?: 'literatura' | 'prontuario';
  }>;
  diagnostics?: {
    retrieval_k?: number;
    used_model?: string; // ex.: 'xgboost_v1' ou 'llm=gpt-4o-mini'
    latency_ms?: number;
  };
}

```

Exemplos completos (cURL e Python)

1) Saúde da API

```
curl -s http://127.0.0.1:8000/health
```

2) Busca semântica pública (literatura)

```
curl -X POST http://127.0.0.1:8000/search
-H "X-API-Key: dev-key" -H "Content-Type: application/json"
-d '{
  "query": "Top 3",
  "k": 3
}'
```

3) Resposta via modelo supervisionado (sem prontuário)

```
python -m src.models.train --cfg ./config/config.yaml

curl -X POST http://127.0.0.1:8000/answer
-H "X-API-Key: dev-key" -H "Content-Type: application/json"
-d '{
  "query": "Top 3",
  "k": 3,
  "class_id": "91",
  "mode": "model"
}'
```

4) Resposta usando RAG + prontuário (glimpses)

Requer `sentence-transformers`, coleção privada populada e papel autorizado.

```
curl -X POST http://127.0.0.1:8000/answer
-H "X-API-Key: dev-key" -H "X-User-Role: psych" -H "Content-Type: application/json"
-d '{
  "query": "Top 3",
  "k": 3,
  "class_id": "91",
  "mode": "model",
  "include_notes": true
}'
```

Boas práticas de uso

- **k pequeno** (3–5) costuma equilibrar contexto e foco; valores maiores podem diluir a resposta.
- `class_id`: use quando a tarefa/modelo for sensível à turma (ex.: ranking por turma).
- **Prontuário** (`include_notes`): ative só quando estritamente necessário e com papel apropriado (princípios LGPD: minimização e necessidade).
- **Citações**: mostre ao usuário final para rastreabilidade.
- **Logs/diagnósticos**: monitore latência e taxa de erro por rota.

Glossário — Siglas e Teorias

- **LGPD**: Lei Geral de Proteção de Dados (Brasil). Princípios: finalidade, adequação, necessidade, transparência, segurança etc.
- **RAG (Retrieval-Augmented Generation)**: técnica que combina **busca de contexto** (retrieval) em base vetorial com um **LLM** para gerar respostas fundamentadas.
- **LLM (Large Language Model)**: modelo de linguagem de larga escala (ex.: GPT). Na API, usado para sintetizar respostas usando os trechos recuperados.
- **Embedding**: representação numérica (vetor) de um texto. Permite medir **similaridade** entre textos.
- **Base Vetorial / Qdrant**: banco especializado em vetores (Qdrant) que armazena embeddings e permite busca por similaridade.
- **Top-k**: os `k` itens mais similares à consulta.
- **Similaridade/Distância**: métricas (ex.: cosseno, dot-product) para comparar vetores. Maior similaridade $\Rightarrow$ mais relevante.
- **Modelo supervisionado**: algoritmo treinado em dados rotulados (ex.: `XGBoost`, `RandomForest`, `Regressão Logística`) para prever/ranquear.
- **SHAP**: técnica para interpretar modelos, atribuindo importância às features de cada predição.
- **Glimpses do prontuário**: janelas/trechos não sensíveis do prontuário usados no retrieval, sob autorização, para enriquecer o contexto.

Observação final: nomes exatos de coleção (`literatura`, `prontuario`), métricas de similaridade e rótulos de modelo (ex.: `xgboost_v1`) podem variar conforme sua configuração em `src/api/main.py` e `config/config.yaml`. Ajuste este documento conforme a versão do seu deploy.